Ruh.ai V1 Tenant Overlay Kit

AI BuildOps - GitHub + Linear + Slack + Sentry

Deployment-ready overlay for one software organization, one trust boundary, and an AI-product delivery conveyor

VERSION	PACK REF	PRIMARY USE	DATE
0.1.0	ai-buildops@1.0.0	Pilot + first production overlay	2026-03-08

Executive snapshot

This overlay kit binds the reusable AI BuildOps pack to a delivery stack where GitHub is the code and pull-request system of record, Linear is the planning system of record, Slack is the operator and approval surface, and Sentry is the normalized source for production incidents and regressions.

The overlay is designed for teams building AI products that already have modern engineering tools but still suffer from fragmented intake, incomplete specifications, inconsistent handoffs, manual release preparation, and slow incident learning loops.

The recommended first activation slice is intake-to-spec, spec-to-build orchestration, eval and release prep, and incident-to-regression learning.

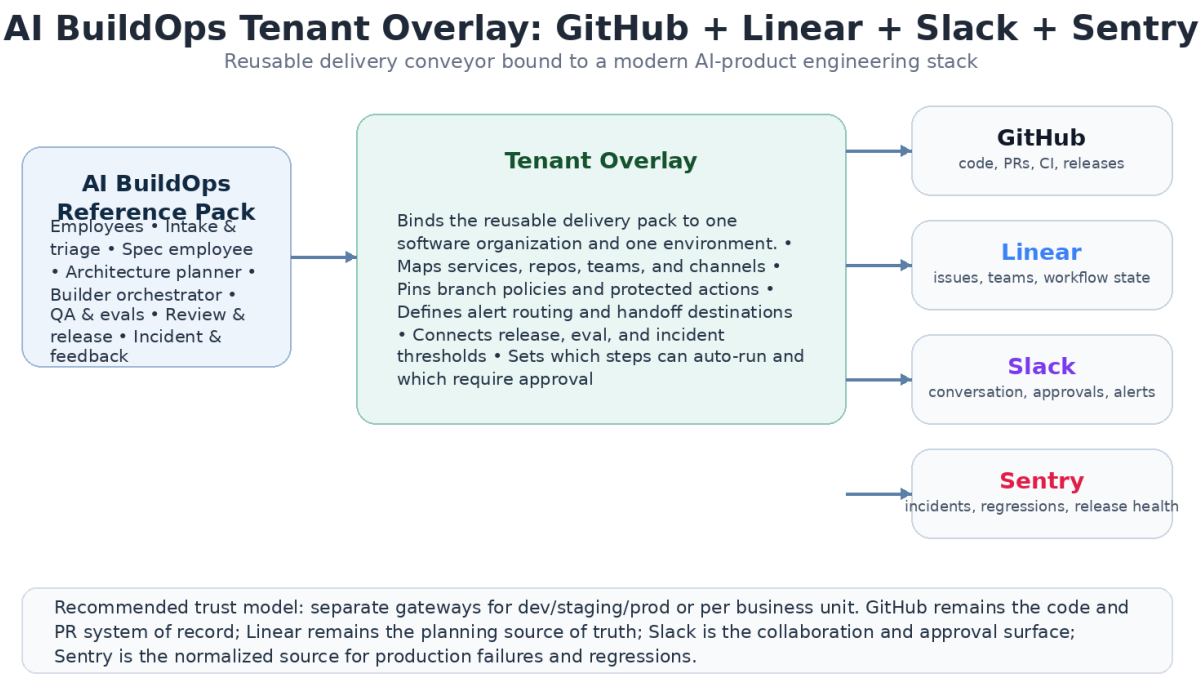

Figure 1. The overlay binds one reusable delivery pack to one engineering stack and one trust boundary.

1. Purpose, scope, and design choices

This document instantiates the AI BuildOps reference pack as a concrete tenant overlay. It specifies how canonical delivery objects map to GitHub, Linear, Slack, and Sentry; which employees are activated; what permissions and approvals apply; what rollout order is recommended; and how a design-partner deployment should be judged before production promotion.

- Keep GitHub as the source of truth for repositories, branches, pull requests, CI status, and releases.
- Keep Linear as the source of truth for issue ownership, planning state, and team routing.
- Use Slack for operator interaction, approvals, digests, and workflow visibility.
- Use Sentry as the normalized production-signal and regression source for post-release learning.

Out of scope in this overlay

Automatic merge to protected branches, autonomous production deployment, or any hidden bypass of existing branch-protection and release controls.

Replacing GitHub, Linear, Slack, or Sentry. The overlay orchestrates around those systems; it does not supersede them.

2. Stack assumptions and tenant prerequisites

The overlay assumes the engineering organization already operates with a reasonably stable repo, planning, and alerting structure. Ruh.ai should remove coordination tax, not compensate for total process absence.

Area	Minimum requirement	Why it matters
GitHub	Organization-level app install, repository list, branch protection already defined	Builder and review workflows need stable repos and PR rules.
Linear	Teams, workflow states, and project taxonomy in active use	Intake and spec handoffs need a clean planning backbone.
Slack	Engineering channels plus a dedicated approvals / alerts surface	Humans need a reliable place to supervise the conveyor.
Sentry	Project-level alerting and release associations in use	Incident learning depends on a trustworthy production signal source.
Service catalog	Named mapping of service -> repo -> team -> on-call group	The overlay routes work by service boundary, not by guesswork.

3. Overlay topology and trust-boundary model

Apply the overlay at the environment boundary first, then parameterize by service. The recommended pattern is one gateway per environment or business-unit trust boundary, with one workspace per service or delivery domain. That keeps repository context, credentials, and incident history from bleeding across unrelated teams.

- Tenant level: engineering-wide policies, connector installations, default evaluation profiles, and role groups.
- Environment level: dev, staging, production, or regulated/trusted segments with separate runtime credentials and deployment rules.
- Service level: repo mapping, Linear team, Slack channel, Sentry project, and approval roles.
- Workspace level: implementation context, generated plans, temporary artifacts, and runtime telemetry for one service boundary.

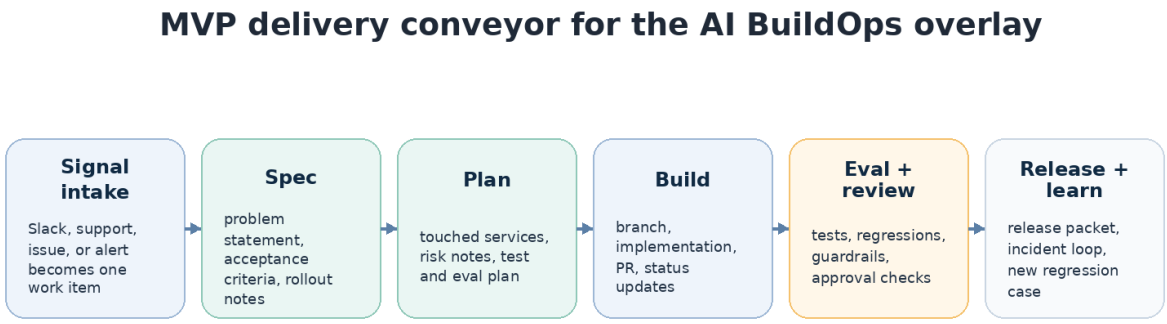

Figure 2. MVP delivery conveyor recommended for the first AI BuildOps overlay rollout.

4. Canonical object mapping

The pack stays horizontal by using canonical delivery objects. Only connectors should understand vendor-specific field names and payload shapes.

Canonical object	Primary source	Secondary source	Notes
WorkItem	Linear issue	Slack or Sentry signal reference	Planning ownership remains in Linear.
SpecArtifact	Ruh.ai artifact	Linked back to Linear issue	The artifact is the auditable planning payload.
CodeChange	GitHub branch / pull request	Linear issue link	Official code state stays in GitHub.
EvalRun	Ruh.ai run artifact	GitHub CI status + Sentry baseline	Eval gate results must be replayable.
ReleasePacket	GitHub release draft / artifact	Slack summary link	Release employee prepares, humans approve.
Incident	Sentry issue / alert	Slack alert thread	Sentry is the primary production-signal source.
RegressionCase	Ruh.ai eval asset	Linked incident / customer transcript	Production failures should feed new regression coverage.

5. Employee activation and permission envelope

The overlay should turn on employees one lane at a time and keep each one inside an explicit permission envelope. This is what makes the digital conveyor governed instead of chaotic.

Employee	Primary actions	Auto allowed	Approval required

Intake & triage	Normalize signals, de-duplicate, create work items	Create Linear issues and Slack summaries	Cross-team escalation outside mapped channels
Spec employee	Draft problem statement, criteria, rollout notes	Create or update structured spec artifact	Major scope change comments back to customer-facing threads
Architecture planner	Map touched services, risks, and test plan	Internal plan artifacts	None by default
Builder orchestrator	Create branch, draft code changes, open PR	Branch creation and PR open	Merge to protected branch or write secrets
QA & evals	Run tests, eval suites, compare baselines	Execute configured eval profiles	Override failed release gate
Review & release	Prepare release packet and docs updates	Internal release draft and changelog	Production deployment or final merge
Incident & feedback	Draft RCA, open follow-ups, create regression cases	Internal incident summary and new eval stub	External incident communication

6. Workflow activation plan

Do not activate the full conveyor on day one. Start with the narrowest slice that crosses the planning, code, and evaluation systems while leaving production control in human hands.

Workflow	Trigger	System of record	Primary metric
Intake -> spec	Slack request, support signal, or incident-linked issue	Linear + Ruh.ai spec artifact	Time from signal to ready-for-build work item
Spec -> build	Approved spec artifact	GitHub PR + Linear state	Spec-to-PR cycle time
Eval + release prep	Ready-for-review PR or release candidate	GitHub CI / release draft + Ruh.ai eval artifact	Gate pass rate and release prep time
Incident -> regression loop	Sentry issue or release regression	Sentry + Ruh.ai regression case	Time from incident to new regression coverage

Recommended pilot order

Week 1-2: intake, work normalization, and spec drafting only, with no builder permissions.
Week 3-4: enable builder orchestration for branch + PR creation in one non-critical service.
Week 5-6: add QA/evals and release packet generation behind explicit approval gates.
Week 7-8: add incident-to-regression loop using Sentry for one production service.

7. Connector profile and secrets inventory

Connector	Required access	Secret reference pattern	Health check
GitHub	Repo read/write for configured automation actions, PR metadata, CI status	secretref://connectors/github/*	List repos + create test branch in sandbox repo
Linear	Issue create/update, team	secretref://connectors/linear/*	Read team map + create dry-

	and workflow read		run issue
Slack	Post messages, receive app mentions/events, create approval prompts	secretref://connectors/slack/*	Post test approval card and receipt
Sentry	Read issues, releases, alert metadata; optional annotations	secretref://connectors/sentry/*	List projects + fetch recent issue
Ruh runtime	Bundle apply, task execution, telemetry upload	secretref://runtime/*	Adapter heartbeat + bundle checksum verification

The overlay should never receive raw credentials in source-controlled files. All tokens and app secrets are injected via the platform secrets broker and referenced only by stable secret IDs.

8. Policy and approval overlay

Build automation becomes dangerous the moment it can silently merge, deploy, or communicate externally. The overlay therefore makes protected actions explicit and records why any gate was overridden.

Risk tier	Examples	Default policy	Approver
Low	Issue creation, spec draft, branch creation, internal digest	Auto allowed	None
Medium	Open PR, update release draft, create regression case in shared repo	Approval required or auto-allowed by service policy	Repo owner or release lead
High	Merge protected branch, deploy production, external incident message	Human only or approval required	Release manager / on-call lead / service owner

- No production deployment action should execute unless the overlay can prove the required eval profile passed and the named approver accepted the action.
- Any missing service mapping, unknown repo, or ambiguous incident ownership should block automation and route to manual triage.
- Every gate override must capture the approver, reason code, and linked work item or incident.

9. Notification, routing, and record ownership rules

Artifact / event	Primary destination	Owner of record
New work item	Linear issue + Slack summary	Linear
Spec artifact	Ruh.ai artifact + link posted in Slack	Ruh.ai artifact linked to Linear
Pull request	GitHub PR + Slack state message	GitHub
Eval summary	Ruh.ai eval artifact + GitHub status context	Ruh.ai eval artifact
Release packet	GitHub release draft + Slack approval prompt	GitHub release draft
Incident summary	Slack alert thread + linked follow-up tasks	Sentry issue + Ruh.ai RCA artifact

Slack is the conversation surface, not the canonical record. The overlay should always post links back to GitHub, Linear, Sentry, or stored Ruh.ai artifacts rather than treating chat messages as the source of truth.

10. Pilot acceptance gates and success metrics

Gate	Target before go-live
Connector auth	All four external connectors pass least-privilege review and health checks.
Routing quality	At least 95% of pilot signals route to the correct service/team in replay.
Eval discipline	100% of release-candidate runs execute the required eval profile.
Protected-action compliance	No merge/deploy action occurs without the required approval record.
Cycle-time impact	Pilot shows measurable reduction in signal-to-spec or spec-to-PR time.

Recommended pilot KPIs

Signal-to-spec turnaround time
Spec-to-PR cycle time
Eval pass rate before release
Incident-to-regression-case turnaround time
Hours saved per engineering manager or tech lead per week

11. Example overlay excerpt

The full starter file is provided separately with this kit. The excerpt below shows the intended structure and the major fields platform and solution engineers are expected to fill.

```
kind: tenantOverlay
apiVersion: ruh.ai/v1alpha1
metadata:
  overlayId: ai-buildops.github-linear-slack-sentry
  packRef: ai-buildops@1.0.0
  displayName: GitHub + Linear + Slack + Sentry overlay
  owner: platform-solutions
  environmentClass: production
  version: 0.1.0
trustBoundary:
  key: product-prod-core
  runtimeProfile: prod
  gateway:
    mode: dedicated
    workspaceStrategy: service-per-workspace
    sandbox:
      mode: non-main
      scope: agent
services:
  catalog:
    - key: api
      repo: github:acme/api
      linearTeam: CORE
      slackChannel: '#api-buildops'
      sentryProject: api-prod
    - key: web
```

```
      repo: github:acme/web
      linearTeam: APP
      slackChannel: '#web-buildops'
      sentryProject: web-prod
connectors:
  github:
    enabled: true
    installationRef: secretref://connectors/github/app-prod
    org: acme
    protectedBranchPolicyRef: main-release-guardrails
  linear:
    enabled: true
    installationRef: secretref://connectors/linear/app-prod
    teamMap:
      CORE: core-platform
      APP: app-experience
  slack:
    enabled: true
    installationRef: secretref://connectors/slack/app-prod
    approvalChannel: '#eng-approvals'
    alertChannel: '#eng-alerts'
  sentry:
    enabled: true
    installationRef: secretref://connectors/sentry/app-prod
```

12. Day-2 operations and change control

- Treat overlay changes like product changes: stage them, test them, and promote them through a controlled environment sequence.
- Keep service mappings versioned. Do not silently repoint one overlay to different repos or teams without an audit trail.
- Rotate app credentials on a regular schedule and verify health checks immediately after rotation.
- Escaped failures should become new regression cases or policy rules before the next overlay release.

Appendix A. Source assumptions

This overlay kit assumes OpenClaw is used as a trust-bounded runtime plane with isolated agents, workspaces, routing bindings, AgentSkills-compatible skill folders, configurable hooks/cron automation, and optional sandboxing. The overlay design also assumes the documented security posture of one trusted operator boundary per gateway, which is why production deployments should separate customers and environments at the gateway layer.

- [R1] Agents - OpenClaw (isolated agents, workspaces, routing).
- [R2] Skills - OpenClaw (AgentSkills-compatible skill folders and precedence).
- [R3] Hooks - OpenClaw (event-driven automations discovered from directories).
- [R4] Cron Jobs - OpenClaw (persistent gateway scheduler).
- [R5] Security - OpenClaw (one trusted operator boundary per gateway).
- [R6] Agent Workspace - OpenClaw (workspace as default cwd; sandbox recommended for isolation).